



南開大學
Nankai University

南 開 大 學

計 算 機 學 院

計算機系統設計實驗報告

PA 3 实验报告

陈语童

年级：2023 级

学号：2311887

专业：计算机科学与技术

课程教师：关浩

2026 年 5 月 26 日

目录

一、 必答题	2
(一) 必答题: 文件读写的具体过程	2
二、 文内思考题	7
(一) Q: 对比异常与函数调用	7
(二) Q: pushl 作用	7
三、 代码实现	8
(一) TODO: 实现 loader	8
(二) TODO: 实现中断机制	9
(三) TODO: 重新组织 TrapFrame 结构体	12
(四) TODO: 实现系统调用	14
(五) CHECKPOINT: PA3 stage1	17
(六) TODO: 在 Nanos-lite 上运行 Hello world (SYS_write 实现)	18
(七) TODO: 实现堆区管理	20
(八) TODO: 让 loader 使用文件	22
(九) TODO: 实现完整的文件系统	24
(十) CHECKPOINT: PA3 stage2	26
(十一) TODO: 把 VGA 显存抽象成文件	27
(十二) TODO: 把设备输入抽象成文件	30
(十三) TODO: 在 NEMU 中运行仙剑奇侠传	32
(十四) CHECKPOINT: PA3 stage3	33
四、 实验环境说明	34
(一) 虚拟机配置	34
(二) IDE 配置	35
五、 总结	36

一、 必答题

(一) 必答题：文件读写的具体过程

问：

仙剑奇侠传中有以下行为：

- 在 navy-apps/apps/pal/src/global/global.c 的 PAL_LoadGame() 中通过 fread() 读取游戏存档；
- 在 navy-apps/apps/pal/src/hal/hal.c 的 redraw() 中通过 NDL_DrawRect() 更新屏幕。

请结合代码解释仙剑奇侠传，库函数，libos, Nanos-lite, AM, NEMU 是如何相互协助，来分别完成游戏存档的读取和屏幕的更新的。

答：

1. 游戏读档

简化流程：

```
PAL_LoadGame()  
-> fopen("/share/games/pal/x.rpg", "rb")  
-> fread(&s, sizeof(SAVEDGAME), 1, fp)  
-> libc 管理 FILE 缓冲  
-> libos: _open/_read/_close  
-> int 0x80 系统调用  
-> Nanos-lite: do_syscall()  
-> fs_open()/fs_read()/fs_close()  
-> file_table 找到 /share/games/pal/x.rpg  
-> ramdisk_read()  
-> memcpy 到 SAVEDGAME 结构体  
-> PAL 做字节序转换并恢复游戏状态
```

说明：程序只调用标准库 fopen() 和 fread()，并不知道底层 ramdisk 的存在。libc 把 fread() 转成底层 _read()，libos 再用 int 0x80 进入 Nanos-lite。Nanos-lite 根据文件名在 file_table 中找到存档文件，利用文件记录中的 disk_offset 和 open_offset 从 ramdisk 复制数据，最终把存档内容读入 SAVEDGAME 结构体。

逐步分析：

仙剑的存档路径写在 navy-apps/apps/pal/include/_common.h：

```
1 #define PAL_PREFIX "/share/games/pal/"  
2 #define PAL_SAVE_PREFIX PAL_PREFIX
```

所以存档文件名会是：

```

/share/games/pal/1.rpg
/share/games/pal/2.rpg
...

```

在 PAL_InitGameData() 中, 如果选择了某个存档槽, 会调用:

```

1 PAL_LoadGame(va("%s%d%s", PAL_SAVE_PREFIX, iSaveSlot, ".rpg"))

```

拼出类似 /share/games/pal/1.rpg 的路径真正的、实际的读取存档操作发生在 PAL_LoadGame():

```

1 fp = fopen(szFileName, "rb");
2 fread(&s, sizeof(SAVEDGAME), 1, fp);
3 fclose(fp);

```

这里 PAL 看到的是标准 C 库接口: fopen()、fread()、fclose()。它不关心 ramdisk、不关心系统调用、不关心 NEMU。库函数负责把这些高级文件操作往下翻译。fopen() 最终会调用 libos 里的 _open():

```

1 return _syscall_(SYS_open, (uintptr_t)path, flags, mode);

```

而 fread() 最终会通过 libc 的 FILE 缓冲机制调用 _read():

```

1 return _syscall_(SYS_read, fd, (uintptr_t)buf, count);

```

这些 _open/_read/_close 都是 libos 提供的“系统调用封装”。它们通过内联汇编执行:

```

1 int $0x80

```

从而把系统调用号和参数放进寄存器, 进入 nanos-lite。

进入内核后, 实现的 trap 中断机制会保存现场, 然后 nanos-lite 的 do_syscall() 中会根据系统调用号进行分发处理:

```

1 case SYS_open:
2     ret = fs_open((const char *)a[1], a[2], a[3]);
3     break;
4
5 case SYS_read:
6     ret = fs_read(a[1], (void *)a[2], a[3]);
7     break;
8
9 case SYS_close:
10    ret = fs_close(a[1]);
11    break;

```

也就是说, 到这一步, PAL 的 fread() 已经变成了 nanos-lite 的 fs_read(fd, buf, len)。nanos-lite 的文件系统维护了一张文件记录表 file_table, 表最后通过 #include "files.h" 包含进 ramdisk 中的普通文件, 而 files.h 里就可以看到 PAL 的资源 and 存档文件:

```

/share/games/pal/sss.mkf
/share/games/pal/data.mkf
/share/games/pal/1.rpg
/share/games/pal/2.rpg
...
/bin/pal

```

`fs_open()` 在 `fs.c` 中线性扫描 `file_table`, 找到路径匹配的文件后返回文件描述符, 这里的文件描述符就是 `file_table` 表的下标。而后 `fs_read()` 在 `fs.c` 中处理普通文件读取, 根据本次实验 FS 部分的代码实现, 对于普通 ramdisk 文件 (所有 PAL 的资源文件、存档文件都是), 这个函数会做:

- (1) 根据 `fd` 找到 `Finfo`
- (2) 检查并裁剪 `len`, 防止越过文件大小
- (3) 用 `file->disk_offset + file->open_offset` 算出 ramdisk 中的真实偏移
- (4) 调用 `ramdisk_read()` 把字节复制到用户缓冲区
- (5) 推进 `open_offset`

这其中的 `ramdisk_read()` 函数, 本质就是 `memcpy(buf, &ramdisk_start + offset, len);`, 将文件拷贝到 `buffer` 中。

综上, 存档读取的核心过程就是:

```

PAL fread(&s)
-> libc FILE 缓冲
-> libos _read()
-> SYS_read
-> Nanos-lite fs_read()
-> ramdisk_read()
-> memcpy 到 PAL 的 SAVEDGAME 结构体

```

读取成功后, `PAL_LoadGame()` 对 `SAVEDGAME s` 做 `DO_BYTESWAP()` 操作, 再把里面的队伍、场景、物品、角色属性等字段搬到 `gpGlobals` 中, 游戏状态就恢复了。在整个过程中 NEMU 的作用是模拟 CPU 和内存, PAL、nanos-lite、ramdisk 都在 NEMU 模拟出的机器里运行。NEMU 不直接理解“仙剑存档”, 只负责绝对服从地执行指令、提供内存; 真正理解文件名、`fd`、`offset` 的是 nanos-lite 的文件系统。

2. 更新屏幕

简化流程:

```

redraw()
-> 把 PAL 的 8-bit vmem 转成 32-bit fb
-> NDL_DrawRect(fb, 0, 0, W, H)

```

```

-> NDL_Render()
-> fseek("/dev/fb", offset)
-> fwrite(canvas row)
-> libc/libos: _lseek/_write
-> int 0x80 系统调用
-> Nanos-lite: fs_lseek()/fs_write()
-> 识别 fd 是 /dev/fb
-> fb_write()
-> AM: _draw_rect()
-> 写入 NEMU 映射的 VGA 显存 0x40000
-> NEMU SDL 更新窗口

```

说明:程序先在用户态把画面缓冲转成 32 位像素,然后交给 NDL_DrawRect(). NDL_DrawRect() 先写到 libndl 的 canvas, NDL_Render() 再按行写入 /dev/fb. 在 Nanos-lite 中, /dev/fb 被当成一个特殊文件处理。write() 不会写 ramdisk, 而是调用 fb_write(), 再通过 AM 的 _draw_rect() 写到 NEMU 的 VGA 显存。NEMU 最后用 SDL 把显存内容显示到屏幕上。

逐步分析:

仙剑的画面更新发生在 hal.c 的 redraw():

```

1 for (int i = 0; i < W; i++)
2     for (int j = 0; j < H; j++)
3         fb[i + j * W] = palette[vmem[i + j * W]];
4
5 NDL_DrawRect(fb, 0, 0, W, H);
6 NDL_Render();

```

PAL 自己维护的 vmem 是 8-bit 调色板索引画面; palette[] 把索引转换成 32-bit RGB 像素。redraw() 先把 vmem 转成真正的 32-bit fb[], 然后交给 NDL_DrawRect() 处理。在非 NWM 模式下, NDL_DrawRect() 并不会立刻系统调用, 而是先把像素复制到 libndl 自己的 canvas 上:

```

1 canvas[(i + y) * canvas_w + (j + x)] = pixels[i * w + j];

```

真正进行写屏幕的是 NDL_Render():

```

1 for (int i = 0; i < canvas_h; i++) {
2     fseek(fbdev, ((i + pad_y) * screen_w + pad_x) * sizeof(uint32_t),
3         SEEK_SET);
4     fwrite(&canvas[i * canvas_w], sizeof(uint32_t), canvas_w, fbdev);
5 }
6 fflush(fbdev);

```

这里的 fbdev 是在 NDL_OpenDisplay() 中打开的:

```

1 fbdev = fopen("/dev/fb", "w");

```

NDL_OpenDisplay() 还通过 get_display_info(); 打开 /proc/dispinfo 文件, 读取 WIDTH 和 HEIGHT 数据, 从而知道 NEMU 屏幕大小, 并计算 pad_x/pad_y, 让 PAL 适应屏幕大小。

这正好对应本次实验实现的两个特殊文件:

- /proc/dispinfo: 把屏幕宽高信息抽象成文件
- /dev/fb: 把 VGA 显存抽象成文件

nanos-lite 在 `init_device()` 里通过 AM 的 `_screen` 生成宽高信息, 然后 `dispinfo_read()` 把宽高信息的字符串当成 `/proc/dispinfo` 的内容返回给用户程序。

实际屏幕写入时, `NDL_Render()` 的 `fseek()` 和 `fwrite()` 会经由 `libc/libos` 接口:

```
fseek()
-> _lseek()
-> SYS_lseek
-> fs_lseek()

fwrite()/fflush()
-> _write()
-> SYS_write
-> fs_write()
```

各个函数不再详细介绍, 代码实现部分中都有详细的代码实现分析。

最后调用 AM 的 `_draw_rect()` 函数, 在 x86-nemu 平台上, 它把像素复制到地址 `0x40000` 开始的 framebuffer 中:

```
1 uint32_t* const fb = (uint32_t *)0x40000;
2 memcpy(&fb[(y + j) * _screen.width + x], pixels, copy_w *
   sizeof(uint32_t));
```

`0x40000` 不是普通地址, 而是通过 `#define VMEM 0x40000` (`vga.c` 中定义) 在 NEMU 里被映射成 VGA 显存。

NEMU 周期性调用 `update_screen()` 真正更新屏幕硬件的显示内容:

```
1 SDL_UpdateTexture(texture, NULL, vmem, SCREEN_W *
   sizeof(vmem[0][0]));
2 SDL_RenderClear(renderer);
3 SDL_RenderCopy(renderer, texture, NULL, NULL);
4 SDL_RenderPresent(renderer);
```

综上, PAL 里的一次屏幕更新所经历的完整过程是:

```
PAL redraw()
-> libndl NDL_DrawRect() 写 canvas
-> libndl NDL_Render()
-> libc fseek/fwrite/fflush
-> libos _lseek/_write
-> SYS_lseek/SYS_write
-> Nanos-lite fs_lseek/fs_write
-> /dev/fb 特判
```

```

-> device.c fb_write()
-> AM _draw_rect()
-> NEMU VGA framebuffer 0x40000
-> SDL_UpdateTexture/SDL_RenderPresent()
-> 屏幕显示仙剑画面

```

二、 文内思考题

PA 在编程指导过程中提出的一些问题，单列出来进行系统的回答。

(一) Q: 对比异常与函数调用

问：我们知道进行函数调用的时候也需要保存调用者的状态：返回地址，以及调用约定 (calling convention) 中需要调用者保存的寄存器。而进行异常处理之前却要保存更多的信息。尝试对比它们，并思考两者保存信息不同是什么原因造成的。

答：

函数调用是“程序主动安排好的控制流”，调用者和被调用者遵守同一套 ABI/calling convention，双方提前约定好哪些寄存器由调用者保存，哪些由被调用者保存。对普通的 call 指令来说，硬件层面主要只需要保存返回地址；剩下的寄存器保存责任由编译器和 ABI 约定完成。异常/中断则不同，它可能发生在任意的指令之后，甚至当前代码完全不知道自己会被打断。异常处理结束后，CPU 必须能像什么都没发生一样回到原位置继续执行。因此不能依赖普通函数调用约定，而要自行保存足够完整、可用于恢复的机器状态。这常常包括但不限于：通用寄存器、EIP、CS、EFLAGS、中断号、错误码等，比函数调用多得多。两者保存信息不同的根本原因在于：**函数调用是事先约定好的局部控制转移，而异常是对不可预测执行现场的强制保存与恢复。**

(二) Q: pushl 作用

问：trap.S 中有一行 pushl %esp 的代码，乍看之下其行为十分诡异。你能结合前后的代码理解它的行为吗？

答：

涉及到的 trap.S 核心代码：

```

1 asm_trap:
2   pushal
3
4   pushl %esp
5   call irq_handle

```

当 pushal 执行完，当前 %esp 正好指向刚刚构造完成的 trap frame 的起始位置。也就是说，此时 %esp 的值就是 _RegSet *。所以 pushl %esp 不是为了保存栈指针本身，而是把 trap frame 的地址作为参数压栈，传给下一步调用的 C 函数：

```

1 _RegSet* irq_handle(_RegSet *tf)

```

pushl + call 两步汇编代码就等价于：


```
1 irq_handle((void *)esp);
```

顺便说明后续 trap.S 中的汇编代码：

```
1  addl $4, %esp
2
3  popal
4  addl $8, %esp
5
6  iret
```

先把刚才压入 irq_handle() 中的函数参数弹出；再通过 popal 依次恢复通用寄存器，并通过 addl 跳过 irq/error_code；最后由 iret 弹出 eip/cs/eflags，回到原用户程序。

所以 pushl %esp 是在把当前栈上的整个 trap frame 的首地址传给 C 语言层，让 irq_handle() 和 do_syscall() 能够借以 _RegSet 结构体读写被中断程序的寄存器现场。

三、 代码实现

本节按照手册指导顺序，根据 TODO 任务切分小节，逐步解析代码实现思路。

(一) TODO: 实现 loader

当前的 loader 只需要将 ramdisk 中从 0 开始的所有内容放置在 0x4000000，并把这个地址作为程序的入口返回即可

```
1 // [PA 3] 补全 loader
2 uintptr_t loader(_Protect *as, const char *filename) {
3     (void)as;
4     (void)filename;
5
6     // 当前 loader 只做最简加载：将整个 ramdisk 搬到固定入口地址。
7     ramdisk_read(DEFAULT_ENTRY, 0, get_ramdisk_size());
8     return (uintptr_t)DEFAULT_ENTRY;
9 }
```

实现的 loader() 函数当前不解析 filename，也不用处理地址空间 as；直接调用提供好的 get_ramdisk_size() 函数获取 ramdisk 总大小，再用 ramdisk_read(DEFAULT_ENTRY, 0, size) 把 ramdisk 从 offset 0 开始的全部内容复制到 0x4000000 (DEFAULT_ENTRY 在上文已经设置为默认入口地址)，最后返回这个地址作为程序入口。补全后 make update 并 make run，得到运行结果如下所示：

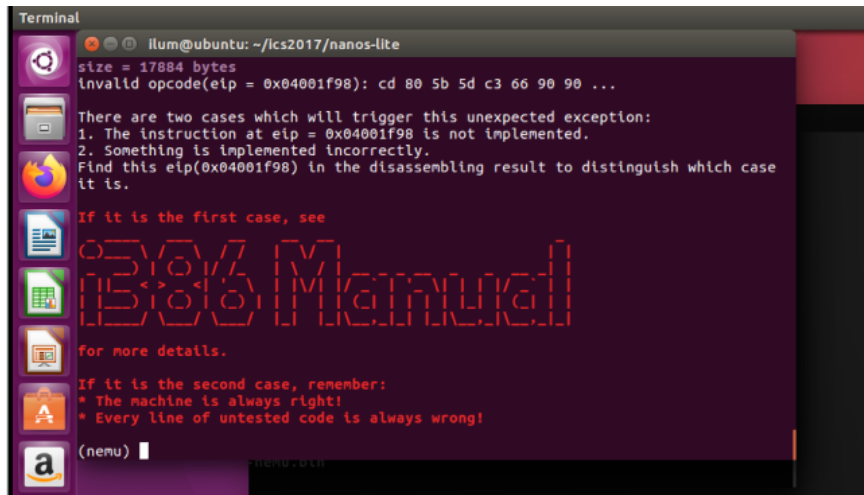


图 1: loader 基本功能验证

如实验手册所说, 会出现一个未实现的 int 指令, 说明 loader 的实现是成功的。

(二) TODO: 实现中断机制

这一部分补全实现执行中断陷入到 trap 中的机制。通过查阅 i386 手册, 得到要实现的两个指令的相关信息:

- lidt: 编码为 0F 01 /3, 操作数是内存中的 6 字节 pseudo-descriptor: limit(16) + base(32);
- int imm8: 编码为 CD 1b, 执行后保存的是下一条指令的 EIP, 并通过 IDTR.base + vector * 8 找 IDT gate。(这里按 PA 要求忽略特权级和完整分段, 只取 gate 的 offset 和 selector)

首先在寄存器中添加了 IDTR 供两个指令使用:

```
1  /* [PA 3] 中断机制的最小状态: LIDT 写入 IDTR, INT 入栈时会用到
   CS。 */
2  struct {
3      uint16_t limit;
4      uint32_t base;
5  } idtr;
6  uint16_t cs;
```

并且在重新启动时对 IDTR 相应位进行初始化:

```
1  static inline void restart() {
2      .....
3      /* [PA 3] 初始运行在 Nexus-AM 使用的平坦内核代码段中。 */
4      cpu.cs = 8;
5      cpu.idtr.limit = 0;
6      cpu.idtr.base = 0;
7      .....
8      /* [PA 3] 中断入口会压入 EFLAGS, 因此复位时保持体系结构要求的 bit
   1 为 1。 */
9      cpu.eflags = 0x00000002;
```

```
10  ....
```

对于两个指令的实现，和 PA 2 中指令添加的流程类似。首先在 exec.c 中分别注册到 opcode 表格中：

```
1  /* [PA 3] 系统指令组中 0f 01 /3 对应 LIDT m16&32。 */
2  make_group(gp7,
3      EMPTY, EMPTY, EMPTY, EX(lidt),
4      EMPTY, EMPTY, EMPTY, EMPTY)
5  ....
6  opcode_entry opcode_table[512] = {
7  ....
8      /* 0xcc */ EMPTY, IDEXW(I, int, 1), EMPTY, EMPTY,      // 0xcd
          对应 INT imm8, 紧随其后的立即数就是中断向量号
9  ....
```

其中，lidt 通过 gp7_E 解 ModR/M 得到内存地址，int 用 IDEXW(I, int, 1) 取 1 字节向量号，之后实现两个指令的 helper 函数：

```
1  /* [PA 3] ASYE/IDT 初始化和系统调用会用到的系统指令。 */
2  make_EHelper(lidt);
3  make_EHelper(int);
4
5  make_EHelper(inv);
6  make_EHelper(nemu_trap);
```

两个指令都属于系统指令范畴，在 system.c 中实例化：

```
1  make_EHelper(lidt) {
2      /* [PA 3] LIDT 操作数格式为 m16&32: 2 字节 limit 后接 4 字节
          base。 */
3      cpu.idtr.limit = vaddr_read(id_dest->addr, 2);
4      cpu.idtr.base = vaddr_read(id_dest->addr + 2, 4);
5
6      print_asm_template1(lidt);
7  }
8  ....
9  make_EHelper(int) {
10     /* [PA 3] INT 保存的 EIP 是 opcode 和 imm8 之后的顺序执行地址。 */
11     raise_intr(id_dest->val, *eip);
12
13     print_asm("int %s", id_dest->str);
14 }
```

其中，lidt 用 vaddr_read() 读 2 字节 limit 和 4 字节 base，写入 cpu.idtr 相应段中。int 调用 raise_intr(id_dest->val, *eip) 进行具体的中断状态中转处理，其中 *eip 已经是中断后的顺序地址。raise_intr() 的实现如下：

```
1  void raise_intr(uint8_t NO, vaddr_t ret_addr) {
2      assert((uint32_t)NO * 8 + 7 <= cpu.idtr.limit);
3  }
```

```

4  /* i386 从 IDTR.base 开始, 按 8 字节门描述符索引 IDT。 */
5  vaddr_t gate_addr = cpu.idtr.base + NO * 8;
6  uint32_t gate_low = vaddr_read(gate_addr, 4);
7  uint32_t gate_high = vaddr_read(gate_addr + 4, 4);
8
9  rtlreg_t val;
10
11 /* 构造 INT 的入栈现场: 依次压入 EFLAGS、CS 和返回 EIP。 */
12 rtl_push(&cpu.eflags);
13 val = cpu.cs;
14 rtl_push(&val);
15 val = ret_addr;
16 rtl_push(&val);
17
18 cpu.cs = (gate_low >> 16) & 0xffff;
19 decoding.jump_eip = (gate_low & 0xffff) | (gate_high & 0xffff0000);
20 decoding.is_jump = 1;
21
22 }

```

首先检查 IDT limit, 用 `vaddr_read()` 读 8 字节 `gate` 进行解析; 然后依次压栈 `EFLAGS`、`CS`、返回 `EIP`; 最后更新 `cpu.cs` 段, 并设置 `decoding.jump_eip/decoding.is_jump` 跳回到中断入口。

上述代码全部实现完备后, 在 `main.c` 中将 `#define HAS_ASYE` 取消注释, 开启 `ASYE` 选项。在 `nanos-lite` 目录下重新 `make update` 并 `make run`, 运行结果如下:

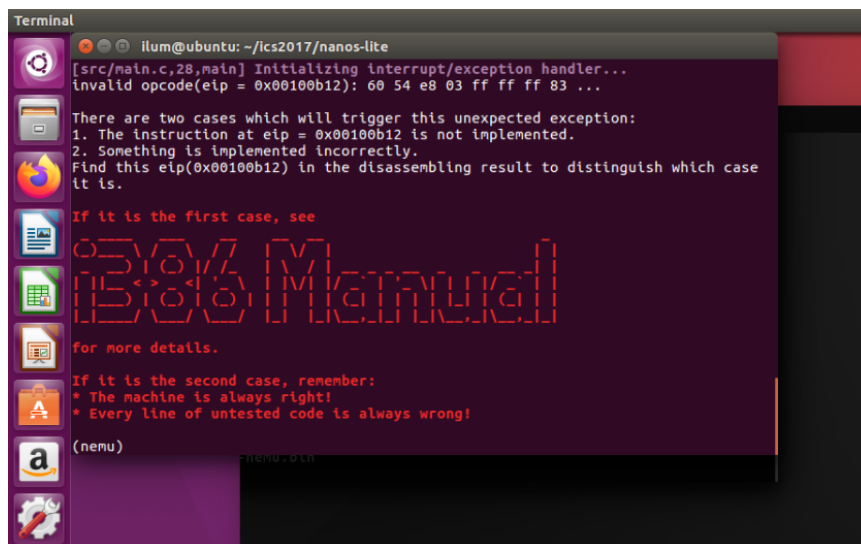


图 2: 中断到达 trap

查看 `nanos-lite` 运行的汇编代码记录, 可以看到:

```

nanos-lite-x86-nemu.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

100b0a: eb 06          jmp 100b12 <asm_trap>

00100b0c <vecnull>:
100b0c: 6a 00          push $0x0
100b0e: 6a ff          push $0xffffffff
100b10: eb 00          jmp 100b12 <asm_trap>

00100b12 <asm_trap>:
100b12: 60             pusha
100b13: 54             push %esp
100b14: e8 03 ff ff   call 100a1c <irq_handle>
100b19: 83 c4 04      add $0x4,%esp
100b1c: 61             popa
100b1d: 83 c4 08      add $0x8,%esp
100b20: cf             iret
100b21: 66 90         xchg %ax,%ax
100b23: 90             nop

```

图 3: 汇编代码中的中断地址

上面运行结果中触发未实现指令的 0x00100b12 对应着 asm_trap 入口地址，与指导手册描述的“在 vecsys()（在 nexum-am/am/arch/x86-nemu/src/trap.S 中定义）附近触发了未实现指令”现象是一致的，详见 trap.S 的实际内容：

```

1 #----|-----entry-----|-----errorcode---|---irq id---|---handler---|
2 .globl vecsys;    vecsys:  pushl $0;   pushl $0x80; jmp asm_trap
3 .globl vecnull;  vecnull: pushl $0;   pushl $-1; jmp asm_trap
4
5 asm_trap:
6     pushal
7
8     pushl %esp
9     call irq_handle
10
11    addl $4, %esp
12
13    popal
14    addl $8, %esp
15
16    iret

```

说明中断机制实现成功。

(三) TODO: 重新组织 TrapFrame 结构体

这一部分需要实现 pusha 指令使得中断进入 trap 后能进行压栈形成 trap frame / 陷阱帧。此外，还需要重新组织 _RegSet 结构体成员，保证和 trap.S 压栈得到的 trap frame 结构一致。

pusha 指令的指令码是 0x60，在 exec.c 的 opcode 表中进行注册，并在 all-instr.h 中声明 helper 函数，不再展示。它的 helper 函数应在 data-mov.c 中实现，需要用到 RTL 的压栈接口 rtl_push() 来构造 trap frame。查阅 i386 手册，pusha/pushad 的执行逻辑是先保存一份入栈前的 sp/esp，再依次压入：

EAX, ECX, EDX, EBX, original ESP, EBP, ESI, EDI

故 pusha 的 helper 实现如下:

```

1 make_EHelper(pusha) {
2     /* [PA 3] pushal 先保存入栈前的 ESP, 再按 i386
       规定的寄存器顺序压栈。 */
3     rtlreg_t old_esp = cpu.esp;
4     rtl_push(&cpu.eax);
5     rtl_push(&cpu.ecx);
6     rtl_push(&cpu.edx);
7     rtl_push(&cpu.ebx);
8     rtl_push(&old_esp);
9     rtl_push(&cpu.ebp);
10    rtl_push(&cpu.esi);
11    rtl_push(&cpu.edi);
12
13    print_asm("pusha");
14 }

```

由于 x86 栈向低地址增长, 所以 pushal 执行完之后, 当前 %esp 指向的内存从低地址到高地址实际排列为:

EDI, ESI, EBP, original ESP, EBX, EDX, ECX, EAX

这指明了, _RegSet 中必须按 edi, esi, ebp, esp, ebx, edx, ecx, eax 布局顺序排列:

```

1 struct _RegSet {
2     /* [PA 3] 成员顺序必须和 trap.S 从当前 %esp 开始的 trap frame
       顺序一致。 */
3     uintptr_t edi, esi, ebp, esp, ebx, edx, ecx, eax;
4     int irq;          // 异常码
5     uintptr_t error_code, eip, cs, eflags;
6 };

```

上面的代码实现完备后, 重新 make update 并 make run, 运行结果如下:

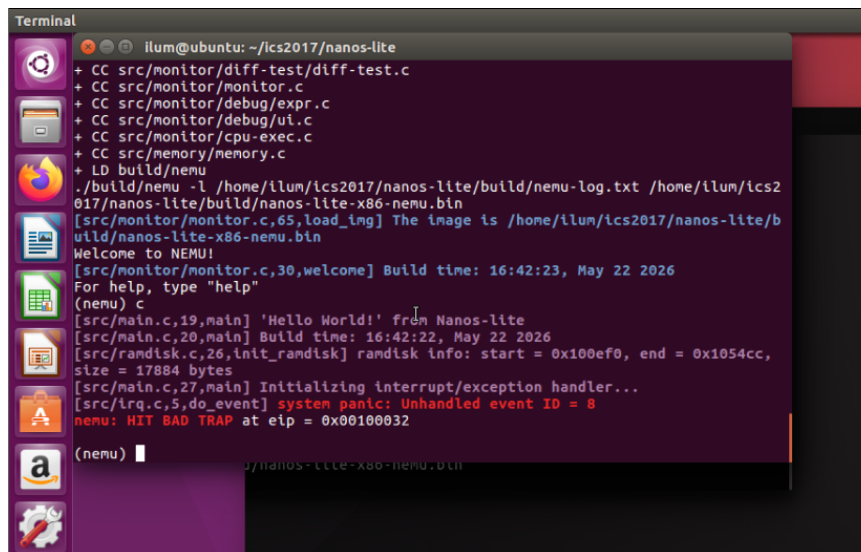


图 4: 重组 TrapFrame 结构体后运行中断

和指导手册中说明的结果一致:

```
[src/irq.c,5,do_event] {kernel} system panic: Unhandled event ID = 8
```

通过 `do_event()` 函数触发了 BAD TRAP. 这说明 `pusha` 指令和 `trap frame` 结构的调整都是正确的。

(四) TODO: 实现系统调用

根据手册的要求逐步实现系统调用的步骤。

1. 识别系统调用事件

`asye.c` 已经把 `irq == 0x80` 翻译成为 `_EVENT_SYSCALL`, 所以 `do_event()` 里新增 `case _EVENT_SYSCALL`, 直接转发到 `do_syscall(r)` 即可。这样不会再在 `irq.c` 里因为 `event 8` 触发 BAD TRAP:

```
1 static _RegSet* do_event(_Event e, _RegSet* r) {
2     switch (e.event) {
3         case _EVENT_SYSCALL:
4             /* [PA 3] 系统调用事件交给 syscall
5              * 分发器处理, 并沿用当前现场返回。 */
6             return do_syscall(r);
7             default: panic("Unhandled event ID = %d", e.event);
8     }
9     return NULL;
10 }
```

2. 实现 SYSCALL_ARGx() 宏

`_syscall_()` 汇编约定是: `eax`= 系统调用号; `ebx/ecx/edx`= 三个参数; 返回值也放在 `eax`。因此宏实现如下:

```

1 #define SYSCALL_ARG1(r) ((r)->eax)
2 #define SYSCALL_ARG2(r) ((r)->ebx)
3 #define SYSCALL_ARG3(r) ((r)->ecx)
4 #define SYSCALL_ARG4(r) ((r)->edx)

```

其中 ARG1 必须能作为左值，因为返回值也要写回它。

3. 添加 SYS_none 系统调用

系统调用号定义本来已在 navy-apps/libs/libos/src/syscall.h 中存在,值为 0。在 do_syscall() 的分发方法中加入 case SYS_none，按照指导手册要求什么都不做，直接返回 1:

```

1 _RegSet* do_syscall(_RegSet *r) {
2     .....
3
4     switch (a[0]) {
5         case SYS_none:
6             /* [PA 3] SYS_none 不执行实际动作，按约定直接返回 1。 */
7             ret = 1;
8             break;
9         default: panic("Unhandled syscall ID = %d", a[0]);
10    }
11
12    .....
13 }

```

4. 设置系统调用返回值

do_syscall() 用局部变量 ret 保存返回值，分发结束后:

```

1 SYSCALL_ARG1(r) = ret;

```

即可设置好返回值，实际上是更改 trap frame 的 eax 值。在回到 trap.S 以后，popal 会把这个 eax 恢复给用户程序。

5. 实现 popa 指令

之前已经实现 pusha 指令。trap.S 中 pushal 后栈顶顺序是 edi, esi, ebp, old_esp, ebx, edx, ecx, eax; 那么 popal 恢复顺序就是弹出 edi/esi/ebp、跳过保存的 old_esp 槽位、再依次弹出 ebx/edx/ecx/eax. 按照上述顺序在 data-mov.c 中编写 helper 函数:

```

1 make_EHelper(popa) {
2     /* [PA 3] popal 按栈帧恢复通用寄存器，但跳过保存的 ESP 槽位。 */
3     rtl_pop(&cpu.edi);
4     rtl_pop(&cpu.esi);
5     rtl_pop(&cpu.ebp);
6     rtl_addi(&cpu.esp, &cpu.esp, 4);
7     rtl_pop(&cpu.ebx);
8     rtl_pop(&cpu.edx);
9     rtl_pop(&cpu.ecx);
10    rtl_pop(&cpu.eax);
11
12    print_asm("popa");
13 }

```


helper 函数的声明和 exec.c 中注册到 opcode 表的操作略。

6. 实现 iret 指令

根据指导手册, 当前实验不处理特权级切换, 所以 iret 只需要弹出三项: eip, cs, eflags. 实现中把弹出的 eip 写入 decoding.jump_eip, 并设置 decoding.is_jump = 1, 让公共更新逻辑跳回用户态的下一条指令即可。在 system.c 中编写 helper 函数:

```
1 make_EHelper(iret) {
2     /* [PA 3] PA 不做特权级切换, iret 只需弹出 EIP、CS 和 EFLAGS。 */
3     rtl_pop(&decoding.jump_eip);
4     rtl_pop(&t0);
5     cpu.cs = t0;
6     rtl_pop(&cpu.eflags);
7     decoding.is_jump = 1;
8
9     print_asm("iret");
10 }
```

helper 函数的声明和 exec.c 中注册到 opcode 表的操作略。

上面步骤都实现完毕后, 再次 make update 并 make run, 运行结果如指导手册预告:

```
Terminal
illum@ubuntu: ~/ics2017/nanos-lite
/home/illum/ics2017/nexus-am/Makefile.compile:86: recipe for target 'klib' failed
make: [klib] Error 2 (ignored)
make[1]: Entering directory '/home/illum/ics2017/nemu'
+ CC src/cpu/exec/system.c
+ CC src/cpu/exec/exec.c
+ CC src/cpu/exec/data-nov.c
+ LD build/nemu
./build/nemu -l /home/illum/ics2017/nanos-lite/build/nemu-log.txt /home/illum/ics2
017/nanos-lite/build/nanos-lite-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/illum/ics2017/nanos-lite/b
uild/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 16:42:23, May 22 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 18:08:53, May 22 2026
[src/randisk.c,26,init_randisk] randisk info: start = 0x100fc0, end = 0x10559c,
size = 17884 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
[src/syscall.c,18,do_syscall] system panic: Unhandled syscall ID = 4
nemu: HIT BAD TRAP at eip = 0x00100032
(nemu)
```

图 5: 实现系统调用过程中测试

——又触发了一个号码为 4 的系统调用。这是一个 SYS_exit 系统调用 (根据 syscall.h 中的 ENUM 枚举可查), 说明之前的 SYS_none 已经成功返回, 此时 dummy 已经执行完毕, 准备退出。

7. 实现 SYS_exit 系统

因此实现中断的系统调用流程还差最后一步, 即实现 SYS_exit 系统调用, 它接收一个退出状态的参数, 用这个参数调用 _halt() 即可:

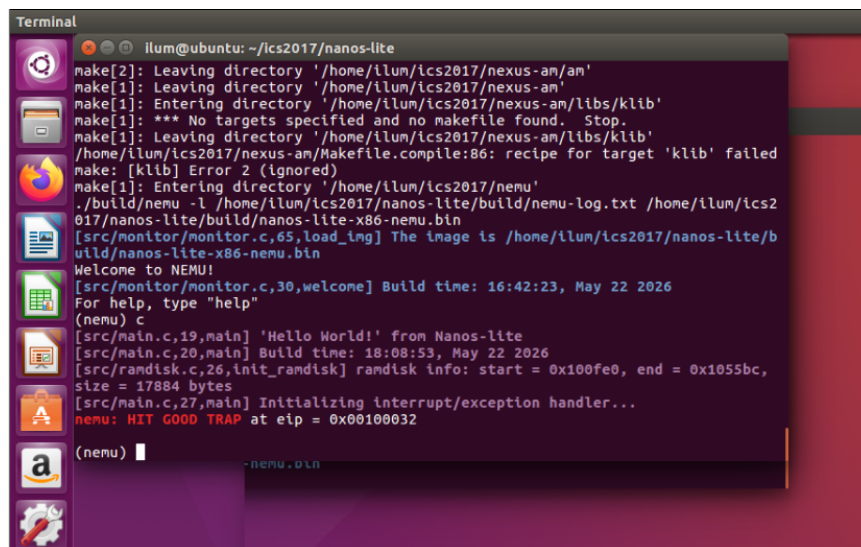
```
1 _RegSet* do_syscall(_RegSet *r) {
2     .....
3
4     switch (a[0]) {
5         .....
```

```

6      case SYS_exit:
7          /* [PA 3] SYS_exit 使用第一个参数作为退出状态并结束当前程序。
            */
8          _halt(a[1]);
9          break;
10         default: panic("Unhandled syscall ID = %d", a[0]);
11     }
12
13     .....
14 }

```

此时再 make update 并 make run, dummy 程序可以到达 GOOD TRAP:



```

Terminal
ilum@ubuntu: ~/ics2017/nanos-lite
make[2]: Leaving directory '/home/ilum/ics2017/nexus-am/'
make[1]: Leaving directory '/home/ilum/ics2017/nexus-am/'
make[1]: Entering directory '/home/ilum/ics2017/nexus-am/libs/klib'
make[1]: *** No targets specified and no makefile found. Stop.
make[1]: Leaving directory '/home/ilum/ics2017/nexus-am/libs/klib'
/home/ilum/ics2017/nexus-am/Makefile.compile:86: recipe for target 'klib' failed
make: [klib] Error 2 (ignored)
make[1]: Entering directory '/home/ilum/ics2017/nemu'
./build/nemu -l /home/ilum/ics2017/nanos-lite/build/nemu-log.txt /home/ilum/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/ilum/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 16:42:23, May 22 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 18:08:53, May 22 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x100fe0, end = 0x1055bc, size = 17884 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
nemu: HIT GOOD TRAP at eip = 0x00100032
(nemu)

```

图 6: 实现系统调用最终测试

说明以上实现都是正确的。现在已经完整实现了中断后的系统调用处理流程。

(五) CHECKPOINT: PA3 stage1

——至此 PA3 stage1 完成。

手动 git 本地记录如下:

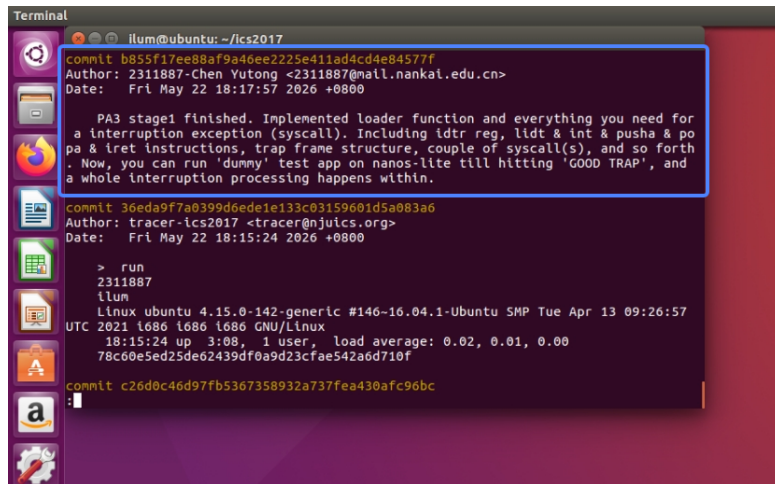


图 7: 手动 git 记录

(六) TODO: 在 Nanos-lite 上运行 Hello world (SYS_write 实现)

首先 man 2 write 查阅相关资料，其给出的核心接口语义是：write 接收 fd、缓冲区地址 buf、字节数 count；成功时返回实际写入的字节数，失败时返回 -1 并设置错误信息。为了实现 SYS_write，在上面实现 syscall 识别和处理的 do_syscall() 内的 switch 分支中添加 SYS_write：

```

1 case SYS_write: {
2     /* [PA 3] SYS_write
3     * write(fd, buf, len): 本阶段只支持
4       stdout/stderr，逐字节写到串口。
5     */
6     int fd = a[1];
7     const char *buf = (const char *)a[2];
8     size_t len = a[3];
9
10    if (fd == 1 || fd == 2) {
11        for (size_t i = 0; i < len; i++) {
12            _putc(buf[i]);
13        }
14        /* man2write 规定成功时返回已写入的字节数，避免 libc
15        认为失败后重试。 */
16        ret = len;
17    } else {
18        /* 当前步骤不实现其它文件描述符，出错约定返回-1。 */
19        ret = -1;
20    }
21    break;
22 }
```

查看 nanos.c 和 syscall.c 中的接口以及 man 2 write 可知，对 write(fd, buf, count) 来说：

- a[0] 是 SYS_write

- a[1] 是 fd
- a[2] 是 buf
- a[3] 是 count/len

所以取参数：

```
1 int fd = a[1];
2 const char *buf = (const char *)a[2];
3 size_t len = a[3];
```

本阶段只处理标准输出和标准错误：fd == 1 是 stdout，fd == 2 是 stderr。由于当前阶段还未复杂文件系统写入逻辑，stdout/stderr 直接映射到最容易观察的串口输出，所以用 _putc(buf[i]) 逐字节输出即可。通过 ret 把返回值写回 eax，具体返回值参考 man 2 write 提供的信息。处理逻辑代码如下：

```
1 if (fd == 1 || fd == 2) {
2   for (size_t i = 0; i < len; i++) {
3     _putc(buf[i]);
4   }
5   /* man2write 规定成功时返回已写入的字节数，避免 libc
6      认为失败后重试。 */
7   ret = len;
8 } else {
9   /* 当前步骤不实现其它文件描述符，出错约定返回-1。 */
10  ret = -1;
11 }
```

补全 SYS_write 后在 nanos.c 中把 _write() 从原来的 _exit(SYS_write) 占位改成真正调用，buf 形参改成 const void *，与 write(2) 的语义一致：

```
1 int _write(int fd, const void *buf, size_t count){
2   return _syscall_(SYS_write, fd, (uintptr_t)buf, count);
3 }
```

上述代码实现都完备后，在 hello 测试的目录下 make 编译，并修改 nanos-lite/Makefile 中 ramdisk 的生成规则，把 ramdisk 中的唯一的文件换成 hello 程序：

```
-- OBJCOPY_FILE = $(NAVY_HOME)/tests/dummy/build/dummy-x86
++ OBJCOPY_FILE = $(NAVY_HOME)/tests/hello/build/hello-x86
```

再在 nanos-lite 目录下重新 make update 并 make run，运行结果如下：

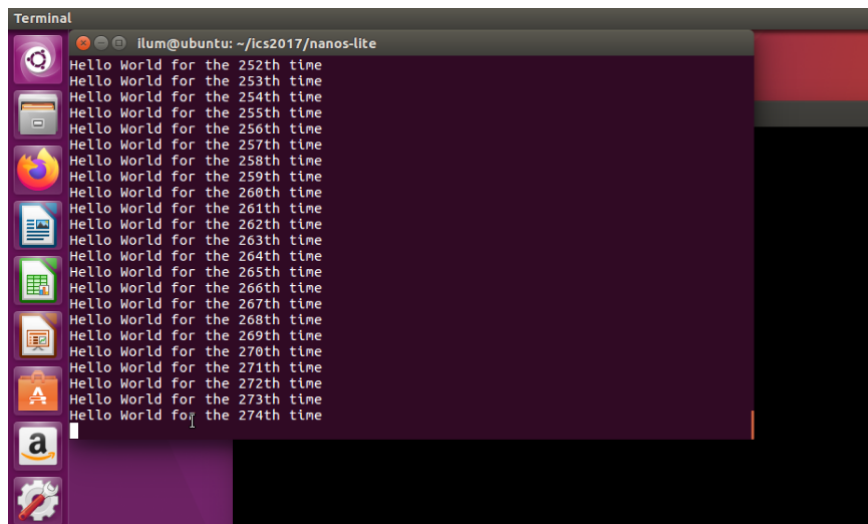


图 8: SYS_write 实现后运行 Hello World 程序

以时钟粒度一直重复输出 Hello World, 和预期效果一致。说明 SYS_write 的实现是正确的。实验手册提到了 printf(), 这里将 write() 和 printf() 进行一个比较。

- write() 是低层接口: 它只负责把一段已经准备好的字节序列写到某个 fd, 而不理解格式化字符串。
- printf() 是库函数: 它负责解析格式串, 比如 %d、\n、字符串拼接、整数转文本等。格式化完成后, 它仍然需要某个底层输出能力把字节真正送出去。

在 newlib 路径里, 可以看到大致链路是:

```
printf() -> vfprintf() -> __sfvwrite() -> __swrite() ->
    _write_r() -> _write() -> SYS_write
```

所以实现 SYS_write 后, hello.c 里直接调用的 write(1, "Hello World!\n", 13) 能输出; 后面循环里的 printf("Hello World for the %dth time\n", i++) 经过格式化后, 也最终能通过 _write() 进入同一个系统调用输出路径, 不断输出 Hello World for...

(七) TODO: 实现堆区管理

man 2 brk/sbrk 中说明了两个关键行为: brk(addr) 用于设置 program break, 成功时返回 0; sbrk(increment) 用于按增量调整 program break, sbrk(0) 可查询当前 break, 成功时返回调整前的旧 program break, 失败时返回 (void *) -1. man 3 end 说明 end/_end 这类符号表示程序各段结束地址, 而且程序需要自己声明这些符号; 也提到有些系统会提供带下划线的 _end 形式。

通过 malloc() 分配堆空间或 Newlib 内部需要堆空间时, 会走到 sbrk(), 再落到 Newlib 的 _sbrk(). _sbrk() 不直接分配内存, 而是维护一个用户态记录的 program_break. 初始 program_break 来自链接脚本符号 _end, 这个符号在 loader.ld 中定义, 其位置在 .bss 之后, 也就是静态数据段结束之后。

其中 `_sbrk(increment)` 会根据当前记录值加上 `increment` 增量得到新的 `break` 地址，它需要系统调用 `SYS_brk`，请求 `nanos-lite` 设置新的 `program break`。根据 `man 2 sbrk` 查询到的信息，如果系统调用返回 0，说明成功，更新 `program_break`，返回旧的 `program break`；反之如果失败，返回 `(void *)-1`。 `_sbrk()` 的实现如下：

```

1 void *_sbrk(intptr_t increment){
2     static char *program_break = NULL;
3
4     if (program_break == NULL) {
5         /* [PA 3] _end 由链接脚本提供，表示用户程序静态数据区之后的初始
6            program break */
7         program_break = &_amp;_end;
8     }
9
10    char *old_break = program_break;
11    char *new_break = program_break + increment;
12
13    if (_syscall_(SYS_brk, (uintptr_t)new_break, 0, 0) == 0) {
14        program_break = new_break;
15        return old_break;
16    }
17
18    return (void *)-1;
19 }

```

没有在 `_sbrk()` 中使用 `printf()`，是为了避免 `printf -> malloc -> sbrk -> printf` 的递归问题

`syscall.c` 中新增对于 `SYS_brk` 的识别和处理。`nanos-lite` 当前只需要返回成功；因此 `SYS_brk` 分支调用 `mm_brk(a[1])`，返回 0：

```

1 case SYS_brk:
2     /* [PA 3] brk(addr): addr 是新的 program
3        break；当前单任务内核直接接受请求 */
4     ret = mm_brk(a[1]);
5     break;

```

`mm_brk` 在 `mm.c` 中定义：

```

1 int mm_brk(uint32_t new_brk) {
2     /* [PA 3]
3        目前没有进程隔离/堆区限制，空闲内存默认交给用户程序按需使用 */
4     (void)new_brk;
5     return 0;
6 }

```

实现完整后，为了检查是否满足“如果你的实现正确，你将会在 `Nanos-lite` 中看到 `printf()` 将格式化完毕的字符串通过一次 `write` 系统调用进行输出，而不是逐个字符地进行输出”这一要求，在 `syscall.c` 中的 `SYS_write` 分支下添加一句 `Log("[Log] SYS_write fd=%d len=%d", fd, len);`，每次调用 `write()` 都输出一行调试信息，从而验证。重新 `make update` 并 `make run` 后结果如下所示：

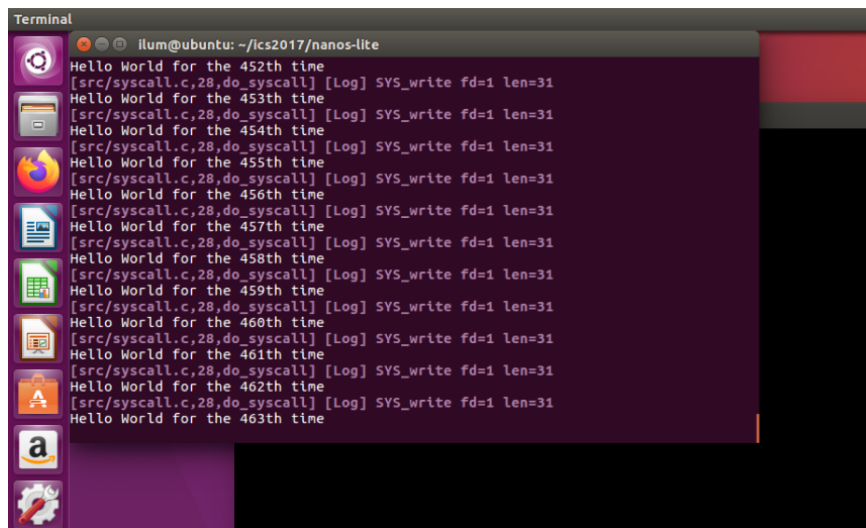


图 9: 堆区支持的批量输出

可以看到，每次输出一句 Hello World 只调用一次 `write()`，说明 `printf()` 是将格式化完毕的字符串通过一次 `write()` 系统调用进行输出的，而非逐字符输出，所以实现正确。

(八) TODO: 让 loader 使用文件

查阅 `man 2 open/read/close` 基本语义，抽取这一步需要的资料，结合 PA 条件和要求可以得到如下函数功能实现的参考：

- `open()` 成功返回一个文件描述符；在这里的简易文件系统里 `fd` 可以直接用 `file_table` 下标表示，`flags/mode` 按要求忽略。
- `read()` 返回实际读到的字节数；如果请求长度超过剩余文件大小，需要只读到文件末尾，并推进当前 `offset`。这里用 `open_offset` 实现这个行为。
- `close()` 成功返回 0；由于当前不维护“打开文件表”状态，所以可以直接返回 0。

首先，按照手册要求在 `fs.c` 里给 `Finfo` 增加了 `open_offset`，用于记录当前文件读指针。在实现实际的 `fs` 功能函数之前，先添加两个小型辅助函数，分别用于在文件表中查找对应文件描述符、返回相应文件的大小：

```
1 /* [PA 3] 从文件表中找到对应文件描述符的记录，并返回指针 */
2 static Finfo* get_file(int fd) {
3     assert(fd >= 0 && fd < (int)NR_FILES);
4     return &file_table[fd];
5 }
6 /* [PA 3] 从文件表中找到对应文件描述符的记录，并返回文件大小 */
7 size_t fs_filesz(int fd) {
8     return get_file(fd)->size;
9 }
```

接下来，就可以在 `fs.c` 中依次实现 `fs_open()`、`fs_read()` 和 `fs_close()`。

对于 `fs_open(pathname, flags, mode)`：遍历 `file_table`，用完整文件名匹配，例如 `/bin/hello`；找到后把 `open_offset` 归零并返回文件表下标作为 `fd`；找不到就 `assert(0)`，后续处理。代码如下：

```
1 int fs_open(const char *pathname, int flags, int mode) {
2     (void)flags;
3     (void)mode;
4
5     for (int i = 0; i < (int)NR_FILES; i++) {
6         if (strcmp(pathname, file_table[i].name) == 0) {
7             file_table[i].open_offset = 0;
8             return i;
9         }
10    }
11
12    assert(0);
13    return -1;
14 }
```

对于 `fs_read(fd, buf, len)`: 先检查 `fd` 合法性; 对 `stdin/stdout/stderr/dev` 这些特殊 `fd` 当前直接返回 0; 普通文件通过 `ramdisk_read()` 从 `disk_offset + open_offset` 读入, 并把读取长度裁剪到文件边界内, 读完推进 `open_offset`. 代码如下:

```
1 ssize_t fs_read(int fd, void *buf, size_t len) {
2     Finfo *file = get_file(fd);
3
4     if (fd < FD_NORMAL) {
5         return 0;
6     }
7
8     assert(file->open_offset >= 0 && (size_t)file->open_offset <=
9         file->size);
10
11    // 简易文件系统中文件大小固定, 读取时不能越过文件边界。
12    size_t left = file->size - file->open_offset;
13    if (len > left) {
14        len = left;
15    }
16
17    // ramdisk 方法读入文件内容
18    ramdisk_read(buf, file->disk_offset + file->open_offset, len);
19    file->open_offset += len;
20    return len;
21 }
```

对于 `fs_close(fd)`: 只检查 `fd` 合法, 然后返回 0, 当前简易文件系统不用维护打开状态。代码:

```
1 int fs_close(int fd) {
2     (void)get_file(fd);
3     return 0;
4 }
```


此前, loader 做的事是“只做最简加载: 将整个 ramdisk 搬到固定入口地址。”而现在 loader 需要根据传入的文件名, 通过 fs 加载文件——只读取被选中的文件, 不再复制整个 ramdisk:

```

1 uintptr_t loader(_Protect *as, const char *filename) {
2     (void)as;
3     int fd = fs_open(filename, 0, 0);
4     fs_read(fd, DEFAULT_ENTRY, fs_filesiz(fd));
5     fs_close(fd);
6     return (uintptr_t)DEFAULT_ENTRY;
7 }

```

这样只需要改 loader(NULL, "/bin/xxx") 传入的文件名, 就能指定加载哪个 ramdisk 文件。

除此之外在 syscall.c 把已经实现的三个 fs 功能函数接到系统调用入口上:

```

1 case SYS_open:
2     ret = fs_open((const char *)a[1], a[2], a[3]);
3     break;
4 case SYS_read:
5     ret = fs_read(a[1], (void *)a[2], a[3]);
6     break;
7 .....
8 case SYS_close:
9     ret = fs_close(a[1]);
10    break;

```

(九) TODO: 实现完整的文件系统

要实现完整的 fs, 还有 fs_write() 和 fs_lseek() 两个函数未实现。

对于 fs_write(): 继续用 _putc() 输出到串口, 保证之前的 printf/write 行为兼容。对于普通文件, 按当前 open_offset 写入 ramdisk, 写入长度会被裁剪到文件边界内, 避免超过固定文件大小, 写完后推进 open_offset。代码如下:

```

1 ssize_t fs_write(int fd, const void *buf, size_t len) {
2     Finfo *file = get_file(fd);
3
4     if (fd == FD_STDOUT || fd == FD_STDERR) {
5         for (size_t i = 0; i < len; i++) {
6             _putc(((const char *)buf)[i]);
7         }
8         return len;
9     }
10
11    if (fd < FD_NORMAL) {
12        return 0;
13    }
14
15    assert(file->open_offset >= 0 && (size_t)file->open_offset <=
        file->size);

```

```
16
17 // 文件大小固定，写入时不能越过原文件边界。
18 size_t left = file->size - file->open_offset;
19 if (len > left) {
20     len = left;
21 }
22
23 ramdisk_write(buf, file->disk_offset + file->open_offset, len);
24 file->open_offset += len;
25 return len;
26 }
```

对于 `fs_lseek()`：支持三种定位方式，分别是从文件开头定位（`SEEK_SET`）、从当前位置继续偏移（`SEEK_CUR`）、从文件末尾开始偏移（`SEEK_END`）。分定位方式实现 `open_offset` 的重计算后，还需要进行越界判断，因为简易文件系统里文件大小是固定的，不能把读写位置移动到文件范围之外。具体处理是，若算出来的位置小于 0，就夹到 0；若算出来的位置大于文件大小，就夹到文件末尾。最后，真正推进 `open_offset`。代码如下：

```
1 off_t fs_lseek(int fd, off_t offset, int whence) {
2     Finfo *file = get_file(fd);
3
4     if (fd < FD_NORMAL) {
5         return 0;
6     }
7
8     off_t new_offset = 0;
9     switch (whence) {
10         case SEEK_SET:
11             new_offset = offset;
12             break;
13         case SEEK_CUR:
14             new_offset = file->open_offset + offset;
15             break;
16         case SEEK_END:
17             new_offset = file->size + offset;
18             break;
19         default:
20             assert(0);
21     }
22
23     if (new_offset < 0) {
24         new_offset = 0;
25     }
26     if ((size_t)new_offset > file->size) {
27         new_offset = file->size;
28     }
29
30     file->open_offset = new_offset;
31     return new_offset;
}
```

```
32 }
```

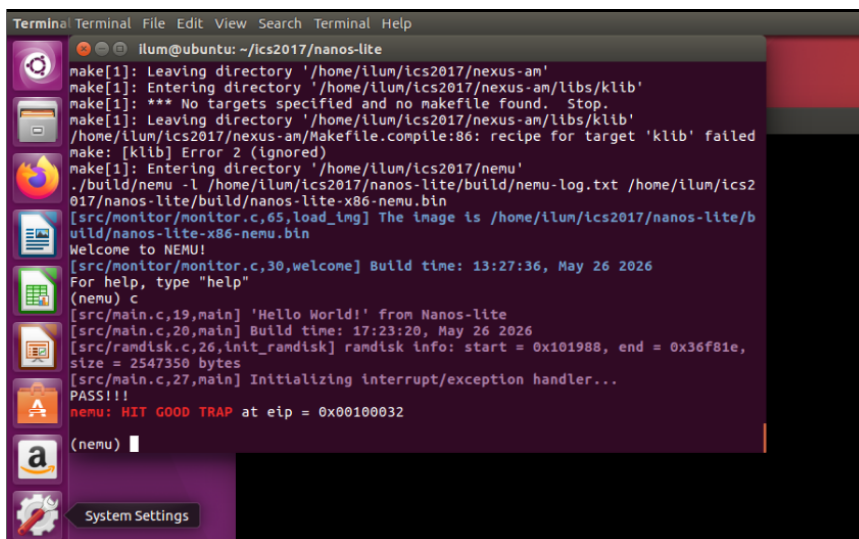
实现上面的两个 fs 函数后, 还需要修改系统调用接口。原先的 SYS_write 只支持 stdout/stderr, 逐字节地写。在实现了 fs_write() 以后, 把 SYS_write 统一交给文件系统处理, 这样普通文件的写入也能正常运作:

```
1 case SYS_write:
2     /* [PA 3] write(fd, buf, len): 统一交给文件系统处理普通文件和
        stdout/stderr。 */
3     ret = fs_write(a[1], (const void *)a[2], a[3]);
4     break;
```

此外, 还需要补充 fs_lseek() 所对应的 syscall:

```
1 case SYS_lseek:
2     /* [PA 3] lseek(fd, offset, whence): 调整文件 open_offset
        并返回新位置。 */
3     ret = fs_lseek(a[1], (off_t)a[2], a[3]);
4     break;
```

上述代码实现完备后, 在 main.c 中调用 loader() 函数处将要装载的用户程序由 /bin/init 切换到 /bin/text, 重新 make update、make run, 运行结果如下:



```
Terminal Terminal File Edit View Search Terminal Help
ilum@ubuntu: ~/ics2017/nanos-lite
make[1]: Leaving directory '/home/ilum/ics2017/nexus-am'
make[1]: Entering directory '/home/ilum/ics2017/nexus-am/libs/klb'
make[1]: *** No targets specified and no makefile found. Stop.
make[1]: Leaving directory '/home/ilum/ics2017/nexus-am/libs/klb'
/home/ilum/ics2017/nexus-am/Makefile.compile:86: recipe for target 'klb' failed
make: [klb] Error 2 (ignored)
make[1]: Entering directory '/home/ilum/ics2017/nemu'
./build/nemu -l /home/ilum/ics2017/nanos-lite/build/nemu-log.txt /home/ilum/ics2
017/nanos-lite/build/nanos-lite-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/ilum/ics2017/nanos-lite/b
uild/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 13:27:36, May 26 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 17:23:20, May 26 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x101988, end = 0x36f81e,
size = 2547350 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
PASS!!!
nemu: HIT GOOD TRAP at eip = 0x00100032
(nemu) |
```

图 10: 实现完整文件系统后的验证

程序输出了“PASS!!!”, 说明用于测试的 /bin/text (简单的文件读写和定位操作) 能够运行通过, 实现的文件系统是正确可用的。

(十) CHECKPOINT: PA3 stage2

——至此 PA3 stage2 完成。

手动 git 本地记录如下:

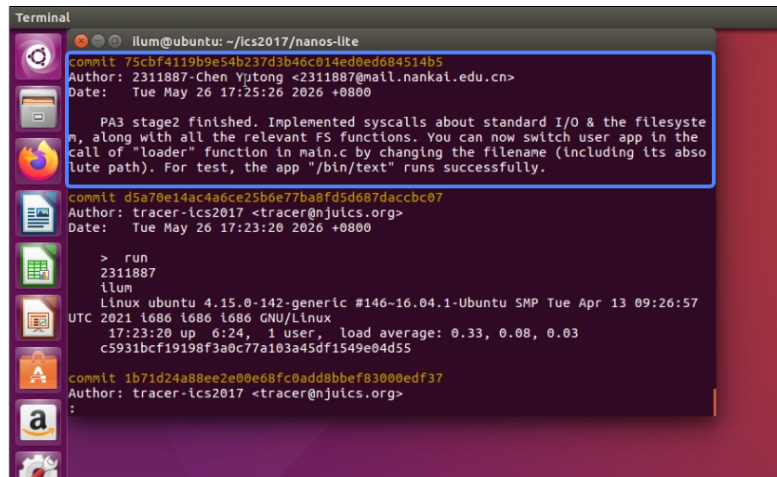


图 11: 手动 git 记录

(十一) TODO: 把 VGA 显存抽象成文件

遵循实验手册中“一切皆文件”的宗旨，需要将各种涉及到的数据结构、信息集合抽象成文件。

第一步，把屏幕信息抽象成 `/proc/dispinfo`。在 `device.c` (line 58) 的 `init_device()` 中，先调用 `_ioe_init()`，然后使用 AM/IOE 提供的 `_screen.width` 和 `_screen.height` 生成文本：

```
WIDTH:400
HEIGHT:300
```

这个格式是 navy-apps 的 libndl 读取 `/proc/dispinfo` 时约定的格式。代码如下：

```
1 void init_device() {
2     _ioe_init();
3
4     /* [PA 3] Navy-apps 的 libndl 会从 /proc/dispinfo 读取
       WIDTH/HEIGHT 两行文本 */
5     snprintf(dispinfo, sizeof(dispinfo), "WIDTH:%d\nHEIGHT:%d\n",
6             _screen.width, _screen.height);
7 }
```

此外还需要实现 `dispinfo_read()`，按照文件系统传入的 `offset` 和 `len`，从这段字符串里拷贝指定范围的数据到用户缓冲区。代码如下：

```
1 void dispinfo_read(void *buf, off_t offset, size_t len) {
2     assert(offset >= 0);
3     assert((size_t)offset + len <= strlen(dispinfo));
4
5     /* [PA 3] /proc/dispinfo
       是内核提前生成的文本信息，读操作只需按文件偏移拷贝 */
6     memcpy(buf, dispinfo + offset, len);
7 }
```

第二步，再把 VGA 显存抽象成 `/dev/fb`。在 `fs.c` 的 `init_fs()` 中初始化 `/dev/fb` 文件大小，使用 IOE 定义的 API 来获取屏幕的大小：

```
1 _screen.width * _screen.height * sizeof(uint32_t)
```

在 `device.c` 的 `fb_write()` 函数中，将文件偏移 `offset` 看成 framebuffer 中的字节偏移，先通过 `offset / 4` 得到像素序号，根据屏幕宽度计算 `(x,y)` 坐标，然后调用 `_draw_rect()` 把像素画到屏幕上。这里采用“每次画一行片段”的方式实现，这样即使一次 `write()` 跨过了行边界，也会拆成多次 `_draw_rect()` 处理。代码如下：

```
1 void fb_write(const void *buf, off_t offset, size_t len) {
2     /* ... */
3
4     /* [PA 3] /dev/fb 按 32-bit 像素线性排列，offset
       先换算成像素序号再换算成屏幕坐标 */
5     const uint32_t *pixels = (const uint32_t *)buf;
6     size_t nr_pixels = len / sizeof(uint32_t);
7     size_t pos = (size_t)offset / sizeof(uint32_t);
8
9     while (nr_pixels > 0) {
10         int x = pos % _screen.width;
11         int y = pos / _screen.width;
12         assert(y < _screen.height);
13
14         size_t row_pixels = _screen.width - x;
15         if (row_pixels > nr_pixels) {
16             row_pixels = nr_pixels;
17         }
18
19         _draw_rect(pixels, x, y, (int)row_pixels, 1);
20         pixels += row_pixels;
21         pos += row_pixels;
22         nr_pixels -= row_pixels;
23     }
24 }
```

第三步，在文件系统中添加对以上实现的两种特殊文件的重定向支持。具体地，在 `fs.c` 中实现如下逻辑：

- `fs_read()` 特判 `FD_DISPINFO`，读取 `dispinfo` 字符串
- `fs_write()` 特判 `FD_FB`，写入 VGA framebuffer
- `fs_lseek()` 放行 `FD_DISPINFO` 和 `FD_FB`，这样 `libndl` 里的 `fseek(fbdev, ..., SEEK_SET)` 能调整 `/dev/fb` 的写入位置

涉及到这两个文件重定向支持的代码是：

```
1 ssize_t fs_read(int fd, void *buf, size_t len) {
2     Finfo *file = get_file(fd);
3 }
```

```
4  if (fd == FD_DISPINFO) {
5      assert(file->open_offset >= 0 && (size_t)file->open_offset <=
        file->size);
6
7      /* [PA 3] /proc/dispinfo 的内容来自 device.c 中预先生成的字符串
        */
8      size_t left = file->size - file->open_offset;
9      if (len > left) {
10         len = left;
11     }
12
13     dispinfo_read(buf, file->open_offset, len);
14     file->open_offset += len;
15     return len;
16 }
17
18 if (fd < FD_NORMAL) {
19     .....
```

```
1  ssize_t fs_write(int fd, const void *buf, size_t len) {
2      .....
3      if (fd == FD_FB) {
4          assert(file->open_offset >= 0 && (size_t)file->open_offset <=
            file->size);
5
6          /* [PA 3] 对 /dev/fb 的写入重定向到
            VGA, 仍然复用文件偏移和边界裁剪语义 */
7          size_t left = file->size - file->open_offset;
8          if (len > left) {
9              len = left;
10         }
11
12         fb_write(buf, file->open_offset, len);
13         file->open_offset += len;
14         return len;
15     }
16     .....
```

其他的读写逻辑和串口输出逻辑不用做更改。

以上代码实现完备后, 在 main.c 的 loader 函数调用处切换加载的程序到 /bin/bmptest, 重新 make update 并 make run, 运行结果如下:

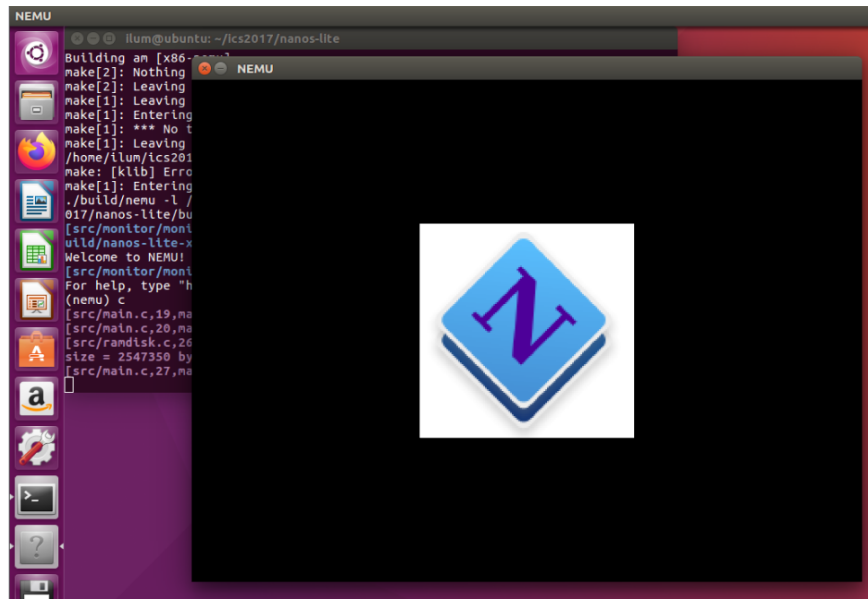


图 12: 在屏幕显示图标

能够成功显示出 ProjectN 的 Logo，说明以上 VGA 显存抽象为文件的实现是正确的。

(十二) TODO: 把设备输入抽象成文件

这一步实现需要把设备输入也抽象为文件。

要读取设备输入,需要在设备层实现函数 `events_read()`。首先调用 AM/IOE 的 `_read_key()` 查询键盘事件, 如果返回 `_KEY_NONE`, 说明当前没有按键事件, 就用 `_uptime()` 生成时间事件; 如果有按键事件, 则解析 AM 约定的 `0x8000` 按下标记——带 `0x8000` 表示按下, 输出 `kd KEY\n`; 不带 `0x8000` 表示松开, 输出 `ku KEY\n`。注意这里的 `KEY` 是来自于文件里已经定义好的 `keyname[]` 字符串数组。在最后, 根据 `len` 的限制做裁剪, 最多拷贝 `len` 字节到 `buf`, 并返回实际拷贝的字节数。函数完整代码如下:

```
1 size_t events_read(void *buf, size_t len) {
2     char event[64];
3     int key = _read_key();
4
5     if (key == _KEY_NONE) {
6         /* 没有按键输入时, /dev/events 暴露当前系统运行时间作为 timer
7            事件。 */
8         snprintf(event, sizeof(event), "t %d\n", (int)_uptime());
9     } else {
10        bool down = (key & 0x8000) != 0;
11        key &= ~0x8000;
12        assert(key > _KEY_NONE && key < (int)(sizeof(keyname) /
13            sizeof(keyname[0])));
14        assert(keyname[key] != NULL);
15
16        /* AM 用 0x8000 标记按下事件, Navy-apps 约定输出 kd/ku 加按键名
17           */
18    }
```

```

15     snprintf(event, sizeof(event), "%s %s\n", down ? "kd" : "ku",
16             keyname[key]);
17
18     size_t event_len = strlen(event);
19     if (event_len > len) {
20         event_len = len;
21     }
22     memcpy(buf, event, event_len);
23     return event_len;
24 }

```

下一步在文件系统中实现对/dev/events的支持。设备输入作为文件被读取, 故在 fs_read() 函数中添加 FD_EVENTS 特判:

```

1     if (fd == FD_EVENTS) {
2         /* /dev/events
3          * 是输入事件流, 没有固定文件大小, 直接交给设备层生成一条事件 */
4         return events_read(buf, len);
5     }

```

/dev/events 是事件流, 而不是 ramdisk 里的固定大小普通文件, 所以这里不使用 open_offset, 也不做文件大小边界裁剪, 直接交给设备层生成一条事件即可。

以上代码均实现完备后, 更改装载的程序为/bin/events, 重新 make update 并 make run, 运行结果如下:

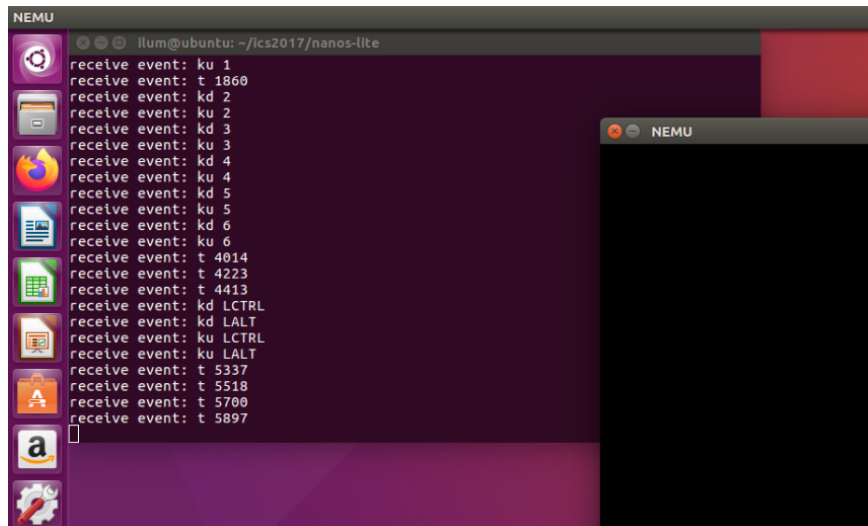


图 13: 设备输入的检测与记录

在没有按键事件发生的时候, 程序连续输出 receive event: t 表示时钟计时; 在 SDL 窗口 (屏幕右侧的黑色窗口中) 按下键盘任意键, 主窗口中则会输出 receive event: kd 和 receive event: ku 并跟上相应按键名, 分别表示按键的按下和抬起动作。如图所示对于按键输入的读取是准确无误的, 说明把设备输入抽象成文件的实现是成功的。

(十三) TODO: 在 NEMU 中运行仙剑奇侠传

按实验手册指导下载“仙剑奇侠传”的数据文件,并放到 navy-apps/fsimg/share/games/pal/目录下。而后在 main.c 中加载 /bin/pal 程序, 重新 make update 并 make run, 运行结果如下:

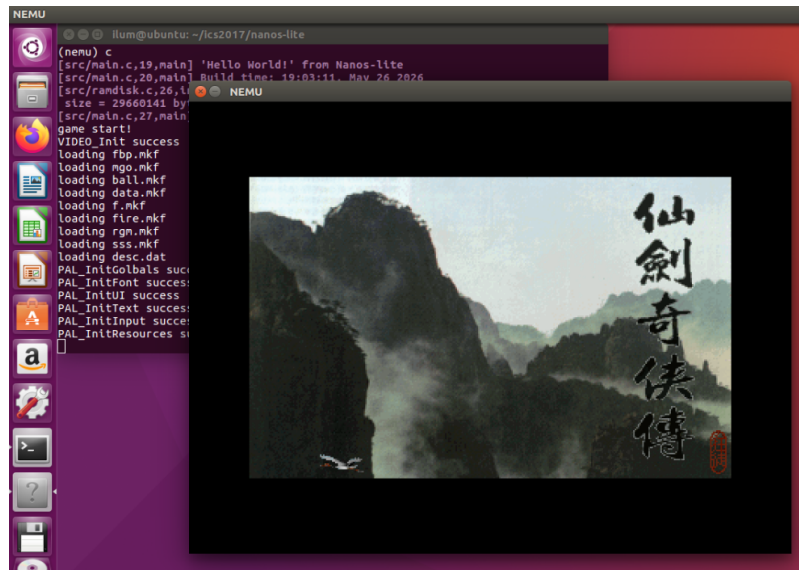


图 14: NEMU 中运行仙剑奇侠传截图 (1)

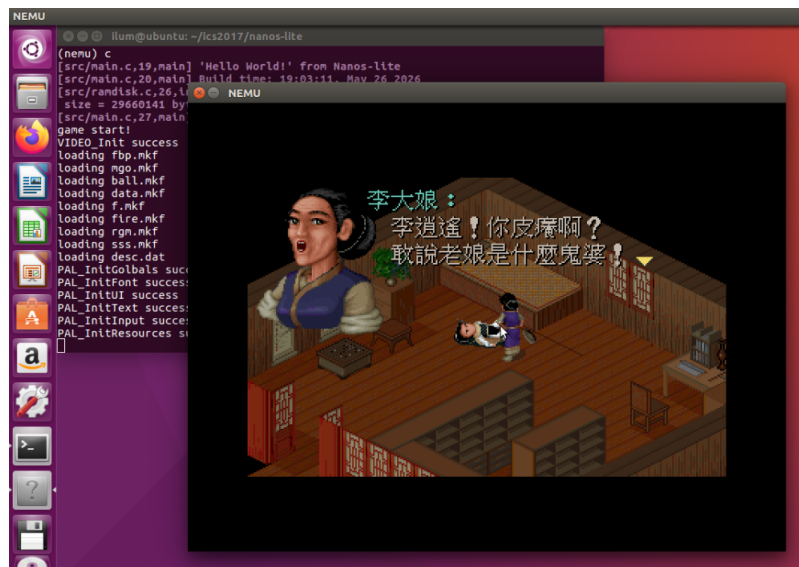


图 15: NEMU 中运行仙剑奇侠传截图 (2)

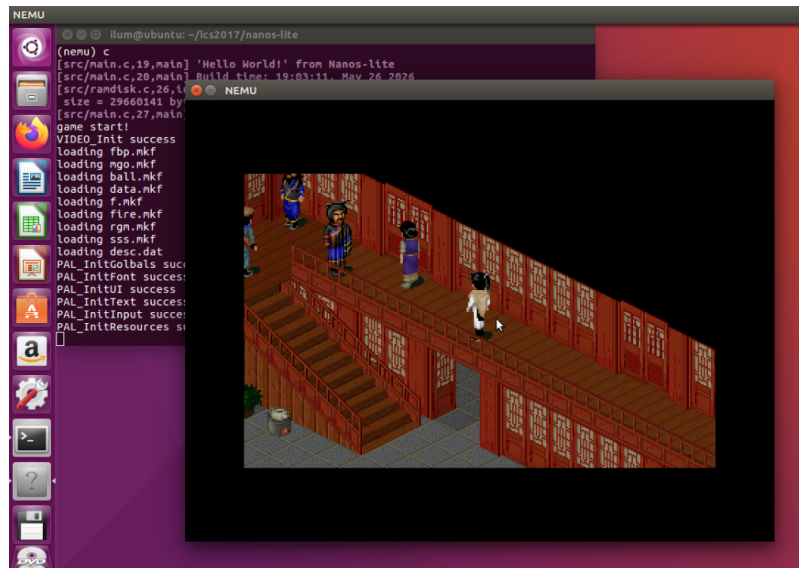


图 16: NEMU 中运行仙剑奇侠传截图 (3)



图 17: NEMU 中运行仙剑奇侠传截图 (4)

经过测试，剑仙游戏能够正常运行，并且敲下数字键和字母键都能够成功读取和相应。
——上述代码实现均正确。

(十四) CHECKPOINT: PA3 stage3

——至此 PA3 stage3 完成。

手动 git 本地记录如下：

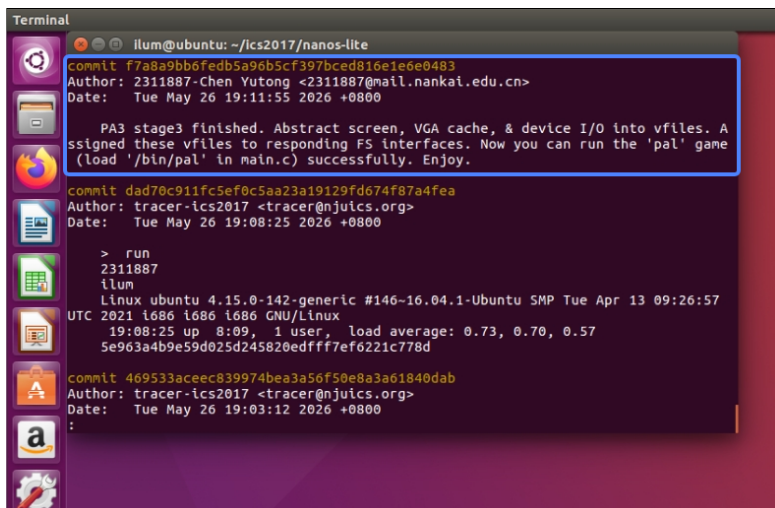


图 18: 手动 git 记录

至此 PA 3 代码实现全部完成

四、 实验环境说明

由于整个课程的实验部分没有要求提交 PA0 / 环境配置一节的报告，此处对实验环境的搭建作简单说明。

(一) 虚拟机配置

采用 VMware 15.5.0 + Ubuntu 16.04 ([ubuntu-16.04.4-desktop-i386.iso](#)) 有可视化界面的虚拟机环境。

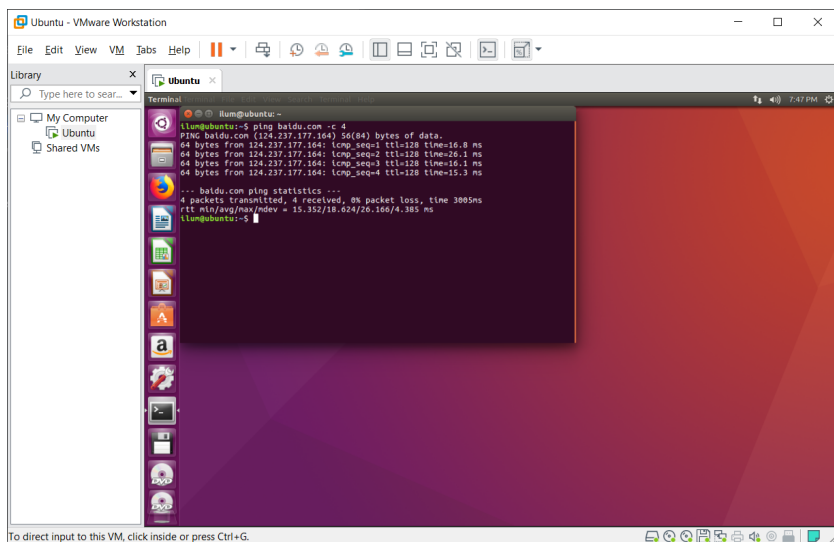


图 19: VMWare + Ubuntu

网络连接无问题。

按照指导书要求，使用 `sudo apt-get install` 命令下载并安装相应的资源工具包。

(二) IDE 配置

此前编程习惯为 vscode 的 IDE 环境，故尝试将此 Ubuntu 虚拟机连接到 vscode。

此前尝试 vscode 的 SSH 功能、VMware 自带的共享文件夹，均失败。

最终使用 SSHFS 的方案，把 Ubuntu 的目录通过 SSHFS 作为一个附加卷挂载到宿主 Windows。安装 WinFsp 和 SSHFS-Win 工具，而后只需要在 Windows cmd 输入命令 `net use X: \\sshfs\[username]@[ip.addr]\[password] /user:[username]`（方括号需要替换为相应内容）在宿主机的文件系统根目录形成一个新的卷轴“X:”，其中内容就是对 Ubuntu 虚拟机指定文件夹的挂载。在 X 盘内右键用 vscode 打开即可用 IDE 环境来编辑代码，同步生效于 Ubuntu 虚拟机。

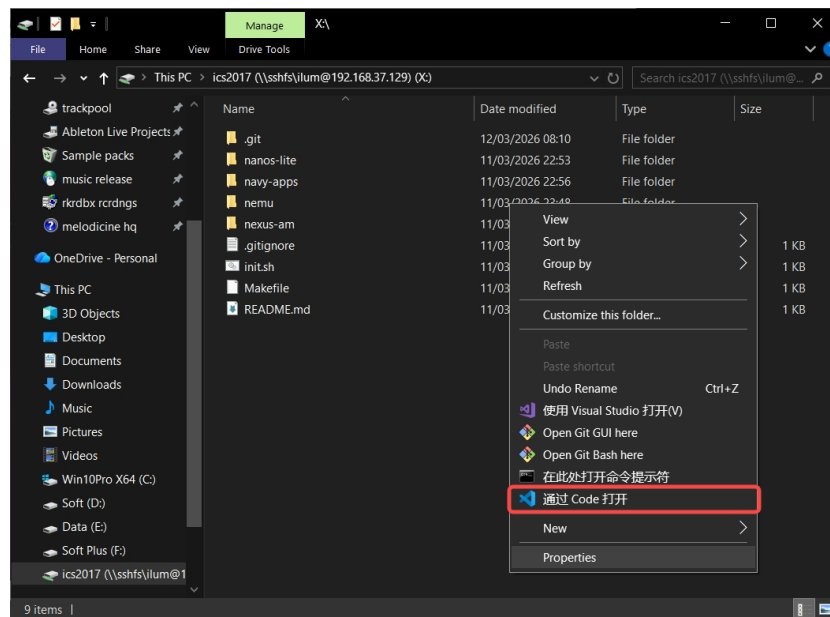


图 20: 本地 vscode 编写代码

五、 总结

本次 PA3 主要补完整了 NEMU 上用户程序运行所需的系统链路:实现 loader 从 ramdisk/文件系统装载程序, 补全 IDT、中断入口、TrapFrame、系统调用返回和 brk; 随后实现文件打开、读写、定位和关闭接口, 并把 /proc/dispinfo、/dev/fb、/dev/events 接入设备层, 最终支撑 Hello World、bmptest、events 以及仙剑奇侠传的读档、绘制和输入。

PA3 实验代码已同步备份至 [GitHub 仓库](#)。